

Configuration Post-Mortem: Wazuh to Telegram SOAR Integration via Shuffle

Mohamed Reda Hamdi , Ibtissam Elouakili

June 6, 2026

Executive Summary

This document outlines the end-to-end troubleshooting and configuration process required to stabilize a Wazuh to Telegram alerting pipeline using Shuffle as the SOAR middleware. The environment suffered from internal Docker routing failures, SSL handshake rejections, blocked execution queues, and UI logic misconfigurations. All issues were resolved, resulting in successful sub-second alert delivery to the target Telegram endpoint.

1 Phase 1: Wazuh Integrator & SSL Evasion

Issue: Wazuh's integrator module inherently drops connections to self-signed HTTPS endpoints. Shuffle's front-end on port 3443 uses self-signed certificates, causing silent failures on webhook transmission.

Resolution: Bypassed the encrypted Nginx proxy and routed the webhook directly to Shuffle's backend API on port 5001 via standard HTTP.

Listing 1: Modifying ossec.conf

```
sudo sed -i 's|https://192.168.2.100:3443|http://127.0.0.1:5001|g' /var  
/ossec/etc/ossec.conf  
sudo systemctl restart wazuh-manager
```

2 Phase 2: Shuffle Execution Engine (Orborus) Routing

Issue: Orborus, the primary execution engine, was trapped in a containerized loop attempting to resolve its backend via `localhost`, resulting in endlessly spinning executions.

Resolution: Updated the Docker environment variables to map the engine to the SIEM VM's physical interface.

Listing 2: Updating OUTER_HOSTNAME

```
sudo nano /opt/Shuffle/.env  
# Modified line: OUTER_HOSTNAME=192.168.2.100
```

```
cd /opt/Shuffle
sudo docker-compose down
sudo docker-compose up -d
```

3 Phase 3: Workflow Logic & Proxy Purging

Issue: Workflows aborted with a “not under startnode” error, and subsequent executions failed silently with no HTTP response body.

Resolution:

1. Connected the Webhook trigger directly to the HTTP execution node within the Shuffle canvas to establish the Start Node hierarchy.
2. Removed a ghost HTTP proxy (192.168.0.1:8080) that was hardcoded in the node settings, preventing outbound API calls to Telegram.

4 Phase 4: Docker Worker Network Timeouts

Issue: Orborus suffered from Worker dispatch timeout errors. It failed to download required execution images in time and could not route worker traffic back to the host via the default Docker bridge.

Resolution: Manually pre-fetched the worker images and forced Orborus to deploy transient workers exclusively onto the internal `shuffle_shuffle` network.

Listing 3: Optimizing Worker Network

```
# Pull required images to bypass timeouts
sudo docker pull frikky/shuffle:worker
sudo docker pull frikky/shuffle:app_sdk

# Force internal networking
echo "SHUFFLE_NETWORK_NAME=shuffle_shuffle" | sudo tee -a /opt/Shuffle/.env
echo "BACKEND_HOSTNAME=shuffle-backend" | sudo tee -a /opt/Shuffle/.env

cd /opt/Shuffle
sudo docker-compose down
sudo docker-compose up -d
```

5 Phase 5: OpenSearch Database Purge

Issue: Hundreds of failed jobs poisoned the OpenSearch execution index with invalid network states, causing Orborus to hang on startup as it attempted to process the dead queue.

Resolution: Executed a direct, authenticated REST API call to OpenSearch to vaporize the entire execution queue, bypassing the frozen UI.

Listing 4: Wiping the Execution Queue

```
curl -k -u admin:StrongShufflePassword321! -X DELETE "https://localhost:9201/*execution*"
sudo docker restart shuffle-orborus
```

6 Phase 6: Final Validation & Payload Formatting

Issue: The Telegram API returns HTTP 400 when malformed Markdown is submitted.

Resolution: Verified payload structure and successfully triggered an LFI payload against the web server, confirming sub-second transmission from Wazuh → Shuffle → Telegram.

Listing 5: Triggering the LFI Alert

```
curl -i "http://localhost/index.html?file=../../../../etc/passwd"
```

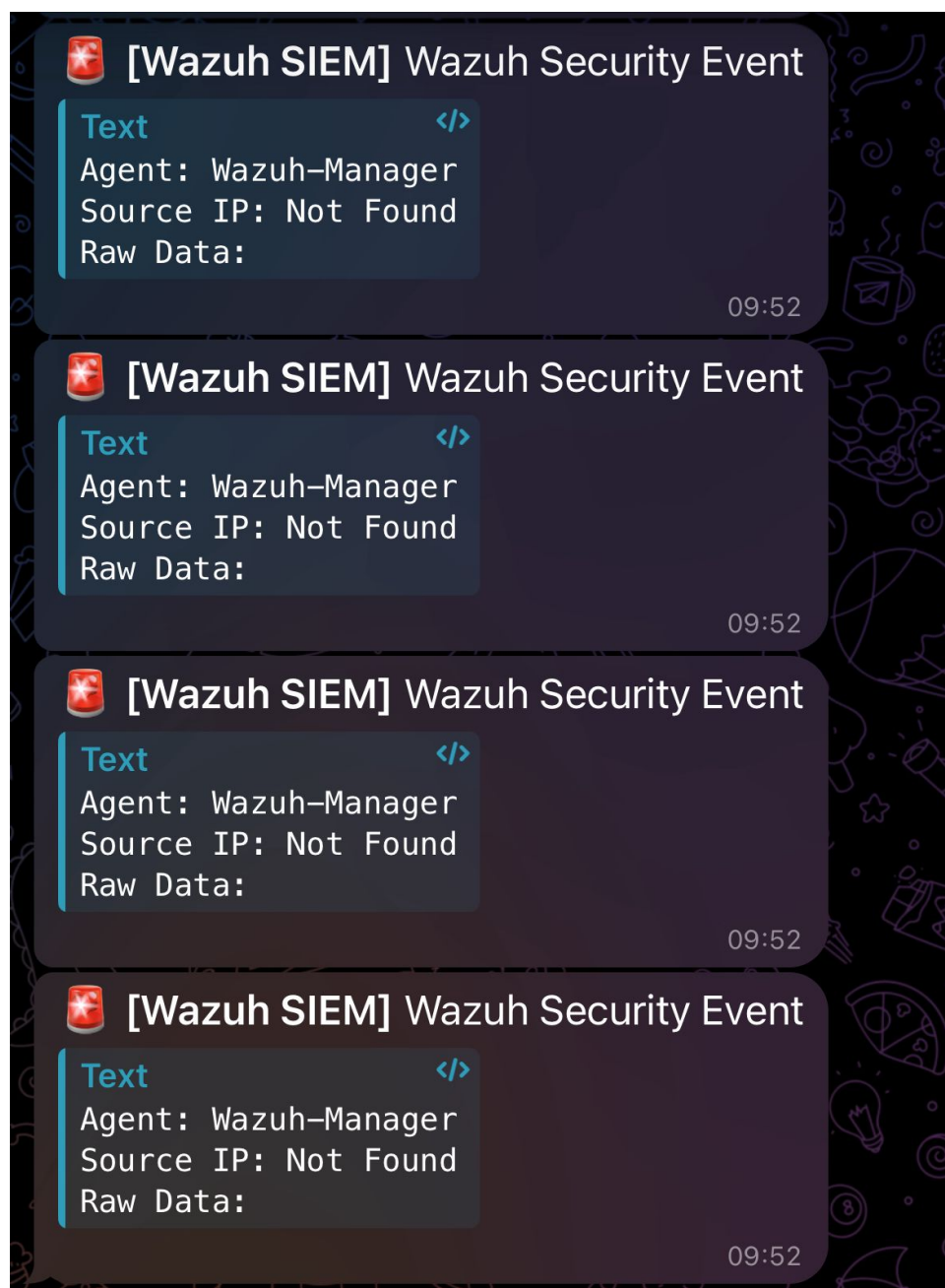


Figure 1: Confirmed delivery of the Wazuh alert in the Telegram client.